

Surrogate modeling of interactions in microbial communities through Physics-Informed Neural Networks.

Paguiel Javan Hossie, Béatrice Laroche, Thibault Malou, Lucas Perrin, Thomas Saigre, Lorenzo Sala

▶ **To cite this version:**

Paguiel Javan Hossie, Béatrice Laroche, Thibault Malou, Lucas Perrin, Thomas Saigre, et al.. Surrogate modeling of interactions in microbial communities through Physics-Informed Neural Networks.. ESAIM: Proceedings and Surveys, 2025, 81, pp.104-122. ⟨10.1051/proc/202581104⟩. ⟨hal-04440736v3⟩

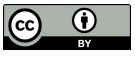
HAL Id: hal-04440736

https://hal.inrae.fr/hal-04440736v3

Submitted on 16 Oct 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

ESAIM: PROCEEDINGS AND SURVEYS, June 2025, Vol. 81, p. 104-122
Didier Auroux, Martin Campos Pinto, Bruno Després, Victorita Dolean, Stéphane Lanteri, Victor Michel-Dansac

SURROGATE MODELING OF INTERACTIONS IN MICROBIAL COMMUNITIES THROUGH PHYSICS-INFORMED NEURAL NETWORKS. *

Paguiel Javan Hossie[1], Béatrice Laroche[2], Thibault Malou[2], Lucas Perrin[3], Thomas Saigre[4] and Lorenzo Sala*[2]

Abstract. Microorganisms form complex communities known as microbiota, influencing various aspects of host well-being. The Generalized Lotka-Volterra (GLV) model is commonly used to understand microorganism population dynamics, but its application to the microbiota faces challenges due to limited bacterial data and complex interactions. This preliminary work focuses on using a Physics-Informed Neural Network (PINN) and synthetic data to build a surrogate model of bacterial species evolution driven by a GLV model. The approach is calibrated and tested on several models differing in size and dynamic behavior.

Résumé. Les micro-organismes forment des communautés complexes, des microbiotes, qui influencent divers aspects de la santé de leur hôte. Le modèle de Lotka-Volterra généralisé (GLV) est couramment utilisé pour comprendre la dynamique des populations de micro-organismes, mais son application au microbiote se heurte à des difficultés dues au manque de données sur les bactéries et à la complexité des interactions. Ce travail so préliminaire se concentre sur l'utilisation d'un Physics-Informed Neural Network (PINN) et de données synthétiques pour construire un modèle approché de l'évolution d'espèces bactériennes décrite par un modèle GLV. Cette approche est calibrée et testée sur plusieurs modèles qui diffèrent par leur taille et leur comportement dynamique.

Introduction

Microorganisms play an essential role as abundant and diverse entities within ecosystems, exerting a significant influence on biological functioning. They often come together in complex communities called microbiota, establishing symbiotic relationships with their environment to maintain a state of equilibrium. However, the spatial distribution of microorganisms in the human body is not homogeneous, varying according to habitat or anatomical site [31]. In the digestive tract, for example, the concentration of microorganisms increases from the stomach, where the acidity and the presence of digestive enzymes are unfavorable for the development of bacteria, towards the colon where conditions are optimal for their growth, the temperature is constant (37 °C), the environment is not very acidic and is rich in water, transit is slow and food is abundant [15]. The gut

* *Corresponding author*

[1] Institut Denis Poisson, UMR 7013 Université d'Orléans et CNRS; e-mail: paguiel-javan.hossie@univ-orleans.fr

[2] Université Paris-Saclay, INRAE, MaIAGE, 78350, Jouy-en-Josas, France e-mail: beatrice.laroche@inrae.fr & thibault.malou@inrae.fr & lorenzo.sala@inrae.fr

[3] INRIA Paris and Laboratoire Jacques-Louis Lions UMR 7598 Sorbonne Université, France; now at Department of Mathematics and Statistics, University of Konstanz, Germany; e-mail: lucas.perrin@uni-konstanz.de

[4] Institut de Recherche Mathématique Avancée, UMR 7501 Université de Strasbourg et CNRS; e-mail: saigre@math.unistra.fr

© EDP Sciences, SMAI 2025

This is an Open Access article distributed under the terms of the Creative Commons Attribution License (https://creativecommons.org/licenses/by/4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

Article published online by EDP Sciences and available at https://www.esaim-proc.org or https://doi.org/10.1051/proc/202581104

microbiota found mainly in the human colon is a dynamic ecosystem whose development is influenced by several factors: genetics, age, geographical location, stress, diet, exposure to infectious agents or pollutants, and antibiotic intake [25, 35]. Numerous studies continue to reveal that the gut microbiota plays a crucial role in various aspects of our well-being [3], such as digestion, regulation of the immune system [18], protection against infection, vitamin synthesis, and even influences on cerebral and metabolic functions [8]. Imbalances or alterations in this ecosystem can be associated with a wide range of health problems, from digestive disorders and autoimmune diseases to obesity and neurological disorders [9]. Certain disturbances in the composition and functions of the microbiota resulting in dysbiosis can lead to certain syndromes such as irritable bowel syndrome [15]. The microbiota can also be used as a medicine (using fecal transplants) to treat certain illnesses such as antibiotic-resistant diarrhea caused by *Clostridium difficile* [15]. In natural ecosystems, the soil microbiota actively participates in the decomposition of organic matter, releasing essential nutrients for plants, and some bacteria can establish symbiotic relationships with plant roots, promoting their growth by providing additional nutrients [12,27]. A better understanding of the microbiota could have a major impact on environmental sustainability, improving human health and managing ecosystems but despite advances in studies of bacterial ecological dynamics, the question of the nature of bacterial interactions in the intestinal microbiota remains open.

The Generalized Lotka-Volterra (GLV) model is commonly used to model and anticipate variations in microorganism populations within an ecosystem [24,36,37]. In the case of the microbiota, it could provide information on how changes in the composition and abundance of a microorganism population could influence the ecosystem [7,16]. Furthermore, by analyzing the model parameters, we can identify the microbial species that have a significant influence on the dynamics of the microbiota as a whole [11,32], predict interspecies interactions [33], species coexistence [14], and even community structure and dynamics [6].

However, when modeling large ecosystems with multiple species, simulating the GLV model may lead to computational challenges. This is particularly true in the context of parameter estimation, where many approaches (for instance Maximum Likelihood or MCMC estimation) explore the parameter space and simulate the model accordingly, potentially generating extensive simulation times and numerically unstable behaviors along the exploration process. Alternative approaches have been proposed in the literature, based on the so-called "metamodelisation" of the system trajectories, for instance through splines [23, 30] or Gaussian processes [10, 38], embedded in a parameter estimation procedure. The main advantages of these approaches are to avoid numerical issues related to dynamical system simulation, and also to allow an easy integration of prior information in the trajectory reconstruction process, such as experimental data. The objective of the present work, in a first step towards parameter estimation, is to investigate the ability of Physics-Informed Neural Networks (PINNs) in providing such metamodels in the specific context of GLV ODEs, based on some typical issues they may raise.

PINNs is a recent and appealing approach that allows the fast and accurate simulation of large and complex dynamical systems [29]. A PINN is a machine learning framework that combines neural networks with physics-based principles to efficiently solve complex physical problems. PINNs are designed to handle supervised learning tasks while respecting prescribed physical laws expressed through partial differential equations (PDEs) or ordinary differential equations (ODEs). They provide a powerful approach to solving problems based on PDEs and ODEs, but training PINN for certain ODE models with either sensitivity to initial conditions, or complex behavior, such as the ones presented above, requires an adequate architecture as well as a significant amount of synthetic training data and computational effort or an accurate emulation [2].

In the present preliminary work, we will design a PINN method, and apply it to the GLV model to simulate the evolution of bacterial species in presence of experimental data, potentially noisy. The paper is organized as follows: in Section 1, we present the governing equations of the GLV model and provide some examples applied to the context of bacterial populations. Section 2 introduces the PINN framework, and presents some preliminary results about the architecture employed in this study. Finally, Section 3 shows the results obtained using the PINN to solve the GLV model, applied to synthetic cases of bacterial populations with and without added noise to the generated observations.

1. Generalized Lotka-Volterra model

In this section, we introduce the Generalized Lotka-Volterra model [36]. This model has been developed to mathematically model the coexistence of various numbers of species in a closed system in a competitive or predator environment. Section 1.1 is devoted to the introduction of the notation and the description of such model, while Section 1.2 presents two examples that will be used in the sequel for numerical applications.

1.1. Description of the model

Assuming large and well-mixed bacteria population, with only bidirectional interactions between bacteria populations, and also assuming that their composition, diversity, and dynamics are not influenced by external factors or environmental conditions (*e.g.* physical and chemical parameters such as temperature, pH, humidity, light intensity, nutrient availability, and oxygen levels, host interactions, ...), then the evolution of N_s different species of bacteria population over a horizon of time $t_{\max}$ can be described by a Generalized Lotka-Volterra (GLV) model [37]:

$$\frac{\mathrm{d}x_i}{\mathrm{d}t}(t) = \mu_i x_i(t) + \sum_{j=1}^{N_s} a_{ij} x_i(t) x_j(t), \quad \forall t \in [0, t_{\max}], 1 \leq i \leq N_s, \tag{1}$$

with the initial condition $x_i(t=0) = x_{i0}$. The scalar $x_i(t)$ represents the abundance of the bacterial population of species i at time t, μ_i represents the intrinsic growth rate of the bacterial population i, and a_{ij} describes the interaction coefficient representing the direct effect of one unit of biomass of the bacterial population of species j on the growth rate of the bacterial population of species i.

Bidirectional interactions among these bacterial populations can be categorized into competition, cooperation, antagonism, and mutualism, but they often lack symmetry due to differences in species characteristics, environmental conditions, and ecological roles. For instance, a negative coefficient a_{ij} means that the population j has a negative impact on the growth of the population i, *e.g.* competition or predation by population j on population i. On the other hand, a positive coefficient a_{ij} means that the presence of the population j enables the population i to thrive, *e.g.* cooperation between the populations i and j or predation by population i on population j.

These asymmetric interactions are crucial for maintaining ecosystem balance; beneficial bacteria can inhibit pathogens through competition and antagonism, while cooperative relationships enhance resilience against infections. However, since a_{ij} and μ_i can take any signs and values, several behaviors can be observed [13, 17, 19, 21] such as oscillations, stable equilibrium with coexisting species, extinction of some species or even demographic explosion or chaos.

The simulation of GLV models can raise numerical issues, to prevent them and avoid unrealistic negative solutions, we assume positive initial conditions and use the logarithmic formulation of the model. For $t \in [0, t_{\max}]$, dividing by $x_i(t)$ in Equation (1) leads to the following formulation:

$$\frac{\mathrm{d}\log(x_i(t))}{\mathrm{d}t} = \mu_i + \sum_{j=1}^{N_s} a_{ij} x_j(t). \tag{2}$$

Setting $\boldsymbol{\mu} = [\mu_1, \cdots, \mu_{N_s}]^T$, $\boldsymbol{A} = (a_{ij})_{1 \leq i,j \leq N_s}$ and $\boldsymbol{u} = [u_1, \cdots, u_{N_s}]^T$ with $u_i = \log(x_i)$, Equation (2) can be written under the matrix form:

$$\frac{\mathrm{d}\boldsymbol{u}(t)}{\mathrm{d}t} = \boldsymbol{\mu} + \boldsymbol{A} \exp\left(\boldsymbol{u}(t)\right), \tag{3}$$

with the initial condition $\boldsymbol{u}(0) = \boldsymbol{u}_0$.

In the following, the matrix $\boldsymbol{\theta}$ denotes the matrix of the GLV parameters, which contains the intrinsic growth rate and the interaction coefficients:

$$\boldsymbol{\theta} = \begin{bmatrix} \mu_1 & a_{11} & \dots & a_{1,N_s} \\ \vdots & \vdots & \ddots & \vdots \\ \mu_{Ns} & a_{N_s 1} & \dots & a_{N_s N_s} \end{bmatrix}.$$

In this study, the classical solver of the GLV model employed as reference is implemented in Python using the `odeint` function from the `scipy.integrate` library. The default solver of the library is used, namely `LSODA`. In the following, we denote by *exact solution*, or *true solution* $\boldsymbol{u}_{\text{truth}}$ the trajectories computed with such solver knowing *a priori* the values of the parameters $\boldsymbol{\theta}$.

1.2. Illustrative cases for numerical simulation of the GLV model

As mentioned above, the solution of the GLV model can exhibit very different behavior. While GLV models with three species or fewer have been extensively studied [19], there are few theoretical results on GLV models in higher dimensions, except for models with specific structures such as bounded competitive GLV (negative off-diagonal terms in the interaction matrix), cooperative systems (positive off-diagonal terms). We refer the reader to [4] for a complete survey of available theoretical results.

If the parameters $(\boldsymbol{\mu}, \boldsymbol{A})$ are taken randomly, the probability to reach a situation where only one species outlives while all the others go extinct increases with the system dimension. Species co-existence through stable oscillations or steady state is indeed non-generic.

A possible approach to obtain an oscillating model consists of generating a matrix $\boldsymbol{A}$ that possesses specific properties:

(i) its diagonal elements and the vector $\boldsymbol{\mu}$ are set to zero, effectively nullifying any self-impact of species;
(ii) the matrix $\boldsymbol{A}$ is antisymmetric, guaranteeing that the interaction between species i and species j is equal in magnitude but opposite in impact to the interaction from species j to species i (predation interaction);
(iii) the sum of each row in $\boldsymbol{A}$ is precisely zero, thereby guaranteeing a compensatory effect for the impact of one species on another.

Assumptions (i) and (ii) ensure the total population to be constant over time (see [4, Chap. 3]), whereas (iii) ensures that vectors with equal, strictly positive coordinates are fixed points of the system. In the case of systems with even dimensions, after [4, Chap. 3], such systems are Hamiltonian and a strictly convex Hamiltonian can be constructed, which guarantees (see e.g. [26]) that the corresponding GLV model exhibits oscillatory dynamics. For systems exhibiting Hamiltonian behavior in even dimensions, exploring the use of symplectic numerical methods [20] provide a structured approach to preserve system properties such as energy conservation.

For the general GLV model, [4, Theorem 5] provides a sufficient condition for the asymptotic convergence towards a stable, coexistence equilibrium, which refers to a situation where the interactions between the different species reach an equilibrium point where they coexist sustainably. Indeed, if the model parameters ensure that the growth rates and interactions among different species balance each other, resulting in a strictly positive equilibrium denoted as $-\boldsymbol{A}^{-1}\boldsymbol{\mu}$, and if there exists a non-negative diagonal matrix $\boldsymbol{D}$ satisfying $\boldsymbol{AD} + \boldsymbol{DA}^\top$ being negative-definite, then the positive equilibrium is both stable and globally attractive in the positive region of the space. This characterization suggests a heuristic to generate such parameters:

(1) Randomly generate a pair $(\boldsymbol{\mu}, \boldsymbol{A}) \in \mathbb{R}^{N_s} \times \mathbb{R}^{N_s \times N_s}$, with $\boldsymbol{\mu} \geq 0$ and the diagonal of $\boldsymbol{A} < 0$, and imposing 20 to 40 % of the extra-diagonal terms of $\boldsymbol{A}$ to be zero.
(2) Solve the equation $\boldsymbol{\mu} + \boldsymbol{AX} = 0$. If any element of $\boldsymbol{X}$ is negative, then go back to step (1), else continue to step (3).
(3) Set $\boldsymbol{D} := \text{diag}(\boldsymbol{X})$, if $\boldsymbol{AD} + \boldsymbol{DA}^\top$ is negative-definite, then keep the generated parameters, else eliminate it and go back to step (1).

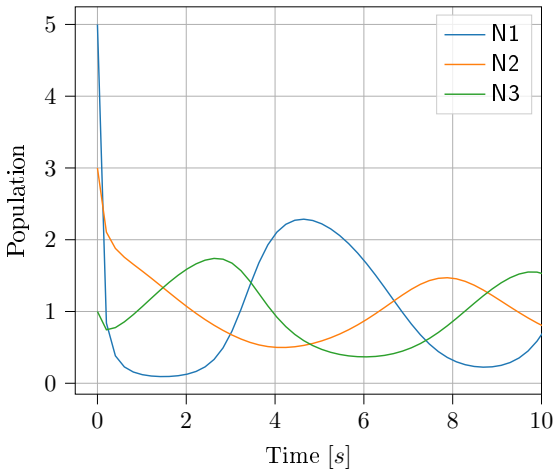
Note that in the limit case where all the extra-diagonal terms of $\boldsymbol{A}$ are zero, the GLV model becomes a set of uncoupled logistic growth equation, with a trivial positive stable steady state satisfying the proposed heuristic. Hence, by playing on the percentage of zeros in the extra-diagonal part of $\boldsymbol{A}$ or on their magnitude, the heuristic should provide adequate parameters, even for high dimensional models.

To illustrate the capability of the PINN to accurately capture the expected outcomes, two illustrative test cases with distinct behaviors will be considered. In this section, the test cases and the related numerical simulations are presented.

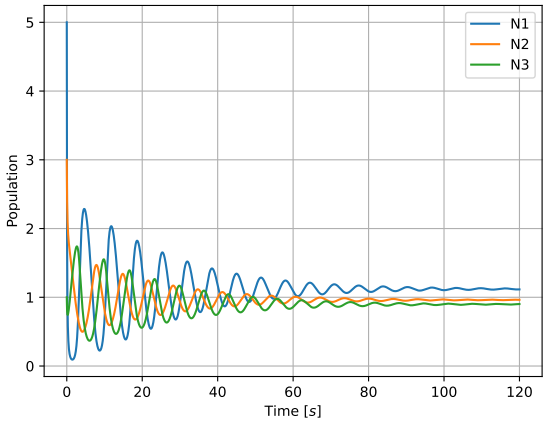
Example 1.1. We introduce a first example with $N_s = 3$ bacterial species. The interaction matrix, the intrinsic growth rate, and the initial population are chosen as follows:

$$\boldsymbol{A}_3 = \begin{bmatrix} -2 & -5 & -0.5 \\ -0.5 & -1 & -1.2 \\ -1 & -0.5 & -1 \end{bmatrix}, \quad \boldsymbol{\mu}_3 = [7.5, 2.6, 2.5]^T \text{ and } \boldsymbol{u}_0 = [5, 3, 1]^T. \tag{4}$$

The growth of one species triggers a reduction in another, fostering a reciprocal cycle until a state of stationary equilibrium is achieved. Figure 1(a) illustrates the evolution of these three populations over the time window of interest for this illustrative example, while Figure 1(b) highlights that stationary equilibrium is achieved in the long term.



(a) Time interval $[0, 10]$ s.



(b) Time interval $[0, 120]$ s.

FIGURE 1. Results of the simulation of the GLV model with three populations of microbes for an initial population $u_0 = [5, 3, 1]^T$.

Example 1.2. We consider a bacterial population with $N_s = 20$ species. Using the algorithm described previously, we generate a parameter matrix $\boldsymbol{\theta}_{20} = (\boldsymbol{\mu}_{20}, \boldsymbol{A}_{20})$ such that the system possess at stationary state $-\boldsymbol{A}_{20}^{-1}\boldsymbol{\mu}_{20}$. The obtained matrix is available at `https://gist.github.com/thomas-saigre/4f92bbb02221a335c4cbafd74b2441fb`. An example of the simulation of the system, using random initial states is presented in Figure 2. The theoretical limits for the populations are also shown. For this case, we select a final simulation time $t_{\max} = 20$ s.

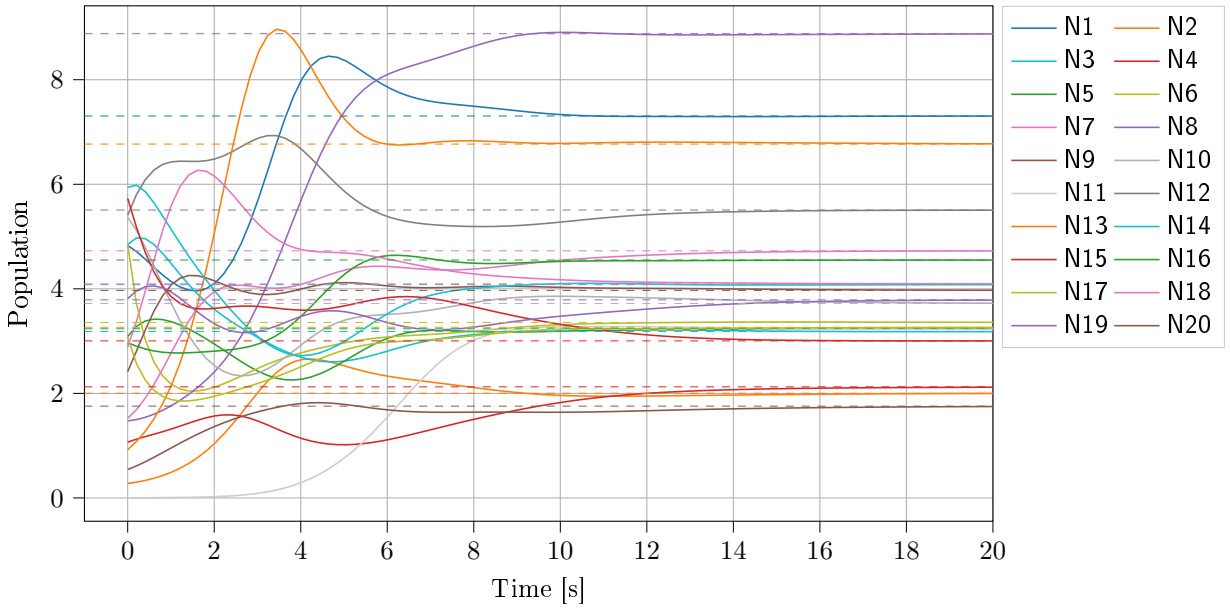


FIGURE 2. Solution of the GLV model with a convergent state, where 20 species are considered, over the time interval $[0, 20]$ s. A random initial condition is considered. The dashed lines represent the theoretical values of the stationary stage $-\boldsymbol{A}_{20}^{-1}\boldsymbol{\mu}_{20}$.

2. Physics-Informed Neural Network

This section introduces the neural network framework that is used to simulate the GLV model: Physics-Informed Neural Networks and the discussion about its architecture.

2.1. PINN framework

Physics-Informed Neural Networks or *PINNs* have been introduced in [29] as neural networks designed to address supervised learning tasks while adhering to specified laws of physics outlined by nonlinear partial differential equations. It combines both supervised and unsupervised learning. In traditional supervised learning, the network learns from a labeled training dataset, where inputs and outputs are matched: the network aims at minimizing a loss function that measures the difference between its predictions and the data labels. In unsupervised learning, the network is exposed to unlabeled data, and its objective is to identify patterns or relationships within the data without explicit guidance from labeled examples. The PINNs are trained to simultaneously minimize the gap between the predictions and the training dataset and satisfy the governing physics equations, thereby incorporating both types of learning to achieve a comprehensive and physics-informed model.

The goal of the PINN is to construct a neural network approximation $\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}(t)$ of the solution $\boldsymbol{u}(t)$ of Equation (3) based on the knowledge of this equation, where $\hat{\boldsymbol{u}}_{\boldsymbol{\theta}} \colon [0, t_{\max}] \to \mathbb{R}^{N_s}$ denotes the function predicted by the network for a given parameter $\boldsymbol{\theta}$. For $t \in [0, t_{\max}]$, we introduce the *residual* $\mathcal{L}$ of the GLV model (3) with respect to the prediction $\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}$ at time t defined as

$$\mathcal{L}(\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}; t) := \frac{\mathrm{d}\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}(t)}{\mathrm{d}t} - (\boldsymbol{\mu} + \boldsymbol{A}\exp(\hat{\boldsymbol{u}}_{\boldsymbol{\theta}}(t))). \tag{5}$$

In the present work, we have in view a specific situation where, unlike in the usual framework of PINNs, the initial condition of the GLV model is not exactly known and it is assumed to be observed together with other observations at other time points. For this reason, we add second information based on a sample of data $\boldsymbol{U}^{(e)}$.

The data, in this specific contribution, are generated numerically, but eventually experimental data should be provided. By averaging measurement errors, introducing these data should reduce the sensitivity to errors on the initial condition and improve the accuracy of our predictions given a fixed grid time points. Thus, the loss function should both satisfy the model (3) and fit the data $\boldsymbol{U}^{(e)}$.

The neural network approach proceeds by using an optimizer to update the weights and bias by minimizing a loss function. In our implementation, we employ the optimizer Adam, implemented in the PyTorch library. This loss function is defined as a linear combination of quantities that measures the quality of our prediction. Specifically in the context of GLV model, we introduce two types of errors for the development of our PINN:

- MSE_{data}: the mean squared misfit by the data, also called the *data loss*, which is used to assess the extent to which the model can faithfully reproduce the data presented

$$MSE_{\text{data}}(t^{(e)}) = \frac{1}{N_s N_{\text{obs}}^e} \sum_{i=1}^{N_s} \sum_{k=1}^{N_{\text{obs}}^e} \left(\hat{\boldsymbol{u}}^i(t_k^{(e)}) - \boldsymbol{U}_{i,k}^{(e)} \right)^2, \tag{6}$$

  for an experiment e, $t^{(e)} = (t_i^{(e)})_{i=1}^{N_{\text{obs}}^e}$ and N_{obs}^e are respectively the time of observations and the number of observations, $\boldsymbol{U}_{i,k}^{(e)}$ represent the quantity of bacterial population of species i observed at time $t_k^{(e)}$ and $\hat{\boldsymbol{u}}_\theta^i$ is the neural network prediction of the bacterial population of species i: $\hat{\boldsymbol{u}} = \left(\hat{\boldsymbol{u}}^i\right)_{i=1}^{N_s}$;
- $MSE_{\mathcal{L}}$: the mean squared residual, also called the *physical loss*, which enforces the structure imposed by (5) at a finite set of collocation points $t_r = \{t_j\}_{j=1}^{N_f} \subset [0, t_{\max}]$

$$MSE_{\mathcal{L}}(t_r) = \frac{1}{N_s N_f} \sum_{i=1}^{N_s} \sum_{j=1}^{N_f} \mathcal{L}_i(\hat{\boldsymbol{u}}; t_j)^2, \tag{7}$$

  where $\mathcal{L}(\hat{\boldsymbol{u}}; t) = (\mathcal{L}_i(\hat{\boldsymbol{u}}; t))_{i=1}^{N_s}$.

Note that the influence of the initial conditions is included in the model, by the loss MSE_{data}, but, as mentioned above, due to potential measurement errors, there is no reason to distinguish it from other available data. We introduce the loss function to be minimized, involving a hyperparameter $\lambda^{\mathsf{PINN}} > 0$:

$$\text{Loss} = MSE_{\text{data}}(t^{(e)}) + \lambda^{\mathsf{PINN}} MSE_{\mathcal{L}}(t_r). \tag{8}$$

For all our numerical experiments, unless otherwise specified, we take $N_f = 100$ collocations points and $N_{obs} = 10$ observational data points. We present in Figure 3 an example of the evolution of the MSEs of the loss, over the iteration during the learning process of the PINN. The loss employing a value of $\lambda^{\mathsf{PINN}} = 0.1$ is also presented. The results presented are obtained from the case with 20 bacterial species, introduced in Example 1.2. We remark that after a certain number of epochs, the global loss decreases steadily, demonstrating the learning process of the PINN, while the data loss stabilizes at a lower value, and the physical loss remains higher, likely due to the complexity of the governing equations.

2.2. PINN architecture

In this section, we discuss the chosen architecture for the neural network trained within the PINN framework exposed above. A single neural network will correspond to a single experiment e, and will therefore be trained to predict, for a time t as input, the population of the N_s species in the given experiment. By adopting a single neural network, the model is tasked with predicting the population dynamics of all N_s species at a given time t within the specific experiment. The rationale behind this decision is to streamline the training process and enhance efficiency instead of considering a different neural network for each species. This consolidated architecture simplifies the complexity of the overall framework and promotes a more unified and manageable

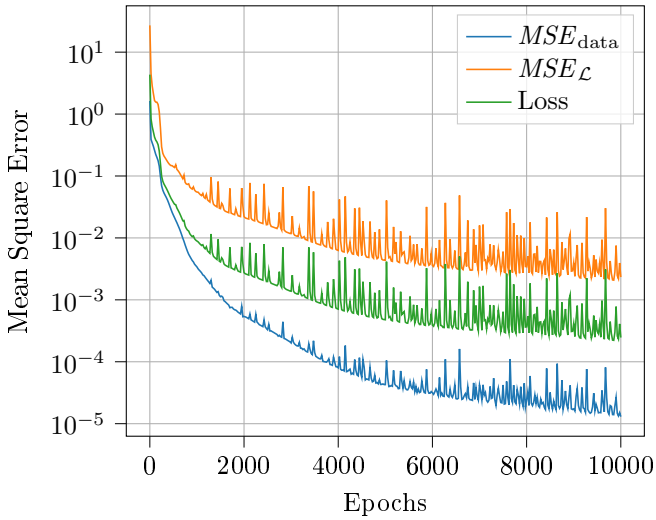


FIGURE 3. Evolution of the data loss MSE_{data} (6), physical loss $MSE_{\mathcal{L}}$ (7) and loss function Loss (8), over the epochs during the training of the PINN for the Example 1.2

training procedure. This strategic choice aimed at achieving a balance between computational efficiency and predictive accuracy, providing a pragmatic solution for the modeling objectives within the PINN framework. Thus, the proposed architecture, outlined in Figure 4, is composed of successive neural layers of various sizes and utilizes the hyperbolic tangent as activation function. The hyperparameters governing the number and size of intermediate layers, denoted as N_{layers} and S_{layers} respectively, are subject to tuning, which is detailed in Section 2.3.

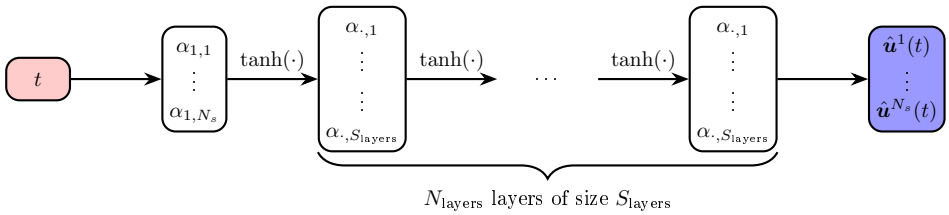


FIGURE 4. Proposed architecture for the PINN framework: an input layer t, a second layer of size N_s, N_{layers} of size S_{layers}, and an output layer of size N_s to predict the population of the different species evaluated at time t. We use a hyperbolic tangent activation function for all our layers.

This chosen architecture aims to strike a balance between model complexity and predictive accuracy, leveraging the interconnectedness of species dynamics while maintaining computational feasibility. The choice of layer sizes being proportional to the number of species N_s allows the network to effectively handle the complexity of interspecies interactions, maintaining a sufficiently rich feature space. Additionally, it is possible to improve the network's ability to capture oscillatory dynamics by incorporating Fourier features [34], which could be particularly beneficial for cases like Example 1.1, where periodic behaviors are prominent. This enhancement will be explored in future work.

In the following, we present two different aspects that have been investigated to increase the effectiveness of the PINN framework, notably the strategy to tune the architecture hyperparameters and the application of time normalization to the classical GLV model.

2.3. **Selection of hyperparameters**

Setting up the PINN described in Section 2.1 requires choosing λ^{PINN} as well as the architecture of the multilayer perceptron (the number of layers N_{layers}, and the size of these layers S_{layers}) and the number of epochs used in training, N_{epochs}, which corresponds to how many times the entire dataset is passed through the neural network during the training process. We decided to use Optuna [1], an open-source hyperparameter optimization framework, to search the hyperparameter space to find the best value for these hyperparameters with respect to a chosen metric. This metric, E_{PINN}, will be the relative error of our prediction concerning the true solution at each collocation point:

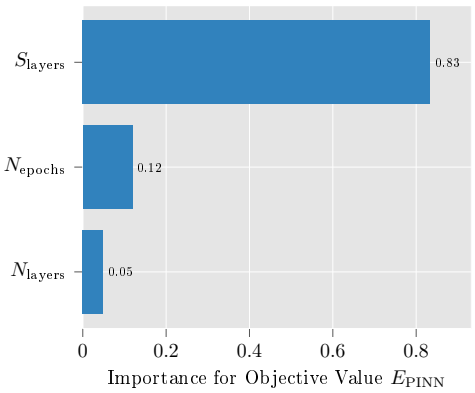
$$E_{\text{PINN}} = \frac{1}{N_s} \sum_{j=1}^{N_s} \frac{\left\| \hat{\boldsymbol{u}}^j - \boldsymbol{u}^j_{\text{truth}} \right\|^2_{L^2([0,t_{\max}])}}{\left\| \boldsymbol{u}^j_{\text{truth}} \right\|^2_{L^2([0,t_{\max}])}}. \tag{9}$$

We explore the tuning of the hyperparameters N_{epochs}, N_{layers} and S_{layers}. The objective value optimized by Optuna, presented in the following results, is the logarithm of the error E_{PINN}. When selecting the architecture of the PINN, we also need to keep in mind the computational cost that would be required by a more complex architecture: the more parameters are involved, which are directly influenced by N_s, N_{layers} and S_{layers}, the more time will be needed to perform the training of the neural network. Unlike the other hyperparameters, we did not perform specific tuning for λ^{PINN}, as its value depends on the confidence in the model versus the data. In this study, λ^{PINN} was chosen deterministically based on prior knowledge, and a thorough exploration of its optimization is beyond the scope of this work.

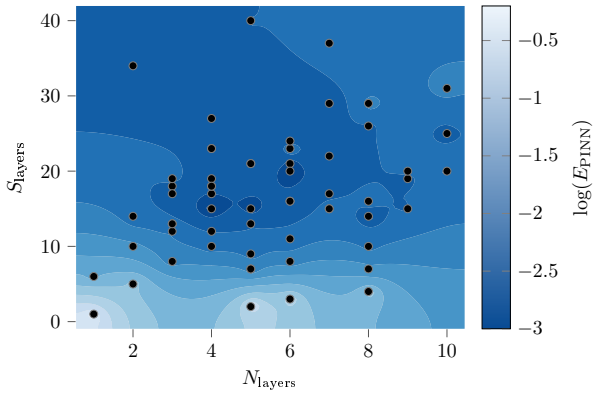
We present in Figure 5 various findings of our investigation performed on the oscillatory test case with $N_s = 3$. Precisely, Figure 5(a) shows the impact of the hyperparameters on the prediction error E_{PINN}, while Figure 5(b) illustrates the error distribution map observed as the size S_{layers} and number of layers N_{layers} vary. As expected, the results indicate that the error is smaller when these two parameters are higher. The interesting finding is that the S_{layers} affects more the relative error than N_{layers}, see Figure 5(a). Moreover, when $N_{\text{layers}} \geq 2$ we do not see such an improvement in the results, this behavior is also notable for $S_{\text{layers}} > 20$. In Figure 5(c) we display the evolution of the prediction error E_{PINN} according to the value of the most influential hyperparameter S_{layers}. We recover the fact that the wider the layers are, the more precise the prediction is and the threshold $S_{\text{layers}} > 20$ where no appreciable increase of accuracy is highlighted. In order to understand the operations carried out by Optuna, we additionally use different colors to illustrate each stage within the trial process. Finally, we performed the same study, focusing on the computational time to find a balance in terms of efficiency for the architecture of the PINN, as presented in Figure 5(d). The impact is measured on two metrics: the prediction error E_{PINN} and the computational time needed to train the neural network, at a fixed number of epochs. We recover that the size of the layers impacts the relative error, but less affects the time of simulation: the training of a deeper neural network will take more time than the training of a wide one.

In light of all these results, we hereafter select the hyperparameters $N_{\text{epochs}} = 2000$, $N_{\text{layer}} = 2$ and $S_{\text{layer}} = 7 \times N_s$, giving the following architecture to the neural network: $[1, N_s, 7 \times N_s, 7 \times N_s, N_s]$. The final number of trainable parameters of the network is then dependant on the number of species. For the two examples presented in this work, with $N_s = 3$ and $N_s = 20$, we have respectively 571 and 25,221 trainable parameters in the neural network.

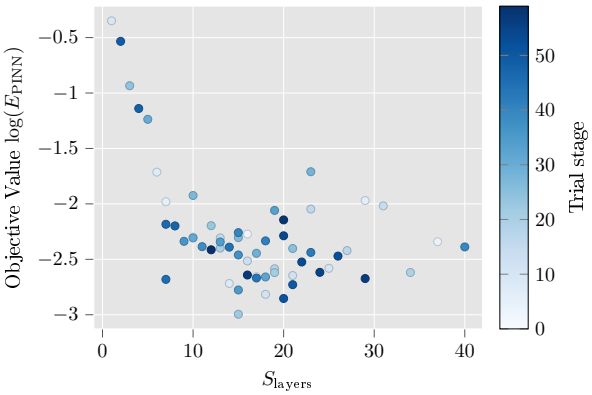
While we utilized Optuna for hyperparameter optimization, our experiments emphasized the more challenging oscillatory dynamics with $N_s = 3$, as this presents a greater challenge for neural networks compared to stationary dynamics. The architecture's proportional scaling with N_s ensures that the increase in the number of species does not impede the network's effectiveness across different system sizes.



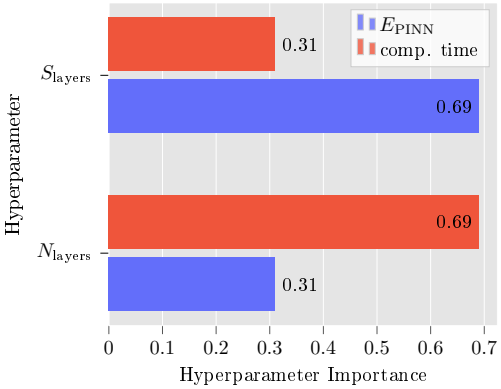
(a) Hyperparameters importance: impact of the architecture of the neural network (numbers N_{layers} and size S_{layers} of the layers), and number of epochs for the training set.



(b) Contour plot showing the distribution of the relative error of the PINN according to its architecture.



(c) PINN relative error against size of layers S_{layers}. The color bar represents the trial stage of Optuna execution.



(d) Effectiveness of the hyperparameters: impact of neural network dimensions on the prediction error E_{PINN} and the computational time.

FIGURE 5. Results obtained with Optuna on the PINN architecture: tuning of hyperparameters: number of layers N_{layers}, layer size S_{layers}, and number of epochs N_{epochs}.

2.4. **Time normalization**

To enhance the stability of numerical computations during the training of PINN, expedite the convergence of PINN learning by mitigating potential issues like vanishing gradients and gradient explosions as discussed in [22, 39], and facilitate the optimization process by improving the handling of diverse scales in model parameters, we propose a modification to the GLV model introduced in Section 1.1. Originally defined over a finite time interval $[0, t_{\max}]$, the model is reformulated by normalizing the time, leading to a formulation defined over a normalized time interval $[0, 1]$. Normalization is a widely adopted approach in machine learning for enhancing neural network performance. In this context, specifically, data contain significant variations from one timescale to another, time normalization would help the model to focus on the underlying patterns (dynamics of competition, predation, cooperation, or other interactions between species that persist over significant time scales despite noise or short-term variations in the data) rather than on variations in amplitude. With this rationale, to obtain the normalized version of the GLV model, we perform the following change of variable $T := t/t_{\max}$. Hence, the

GLV model (3) is rewritten as:

$$\frac{\mathrm{d}\boldsymbol{u}(T)}{\mathrm{d}T} = t_{\max}(\boldsymbol{\mu} + \boldsymbol{A} \cdot \exp(\boldsymbol{u}(T))) \quad \text{for } T \in [0,1]. \tag{10}$$

This change of variable leads to a change also in the loss function (8), specifically in the physical loss $MSE_{\mathcal{L}}$ (7) leading to change of the hyperparameter λ^{PINN}:

$$\lambda^{\mathsf{PINN}}_{\text{normalized}} = \lambda^{\mathsf{PINN}}(t_{\max})^2. \tag{11}$$

On Figure 6, the PINN predictions $\hat{\boldsymbol{u}}$ obtained with the original GLV model (3) and with the normalized GLV model (10) using the appropriate scaling of λ^{PINN} are compared for both cases introduced in Section 1.2.

These results suggest that the prediction is better when time normalization is used, confirming our hypotheses. Specifically, for the oscillatory test-case presented, see Figure 6(a), the mean relative error on the predicted trajectories (9) is $E^{\text{w/o norm}}_{\text{PINN}} = 0.14$ without the normalization and $E^{\text{w/ norm}}_{\text{PINN}} = 2.04{\cdot}10^{-2}$ with the normalization. For the second test case with $N_s = 20$ species, see Figure 6(b), even though it is less striking than in the oscillatory case, the time normalization enables to improve the prediction accuracy. The mean relative error on the predicted trajectories is $E^{\text{w/o norm}}_{\text{PINN}} = 0.11$ without the normalization and $E^{\text{w/ norm}}_{\text{PINN}} = 7 \cdot 10^{-2}$ with the normalization.

We figure that with the time normalization, the error gains one order of magnitude. Hence, we will consider the normalized version of the PINN for further simulations. Our tests indicate that logarithmic normalization of concentrations does not significantly increase training duration while improving the accuracy of the results, making it a suitable choice for our analysis. Hereafter, by abuse of notation, t will refer to the normalized time variable and λ^{PINN} to the normalized hyperparameter $\lambda^{\mathsf{PINN}}_{\text{normalized}}$.

3. Numerical results over the influence of data

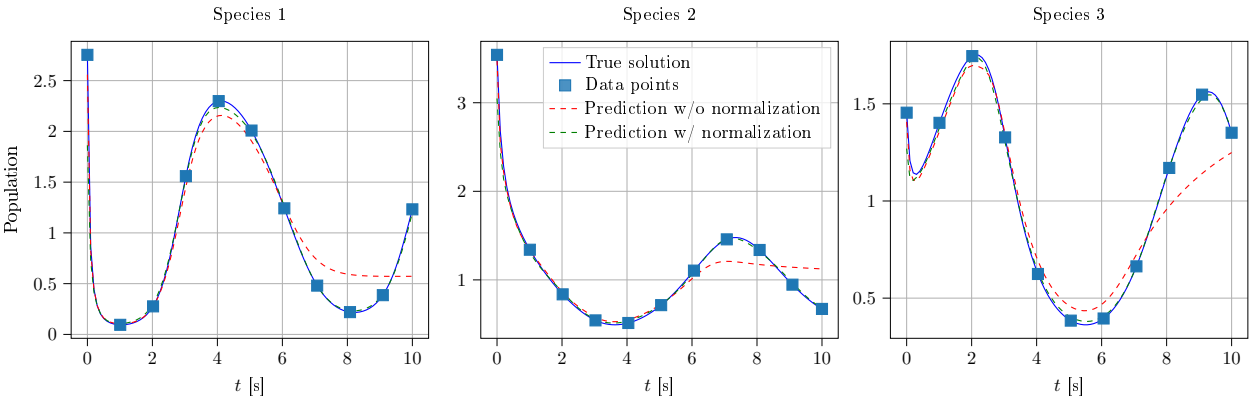
This section presents some numerical results obtained with the PINN described in the previous section. The cases introduced in Examples 1.1 and 1.2 will be both used in the following to compare $\hat{\boldsymbol{u}}$ the approximation computed by the PINN with the reference solution $\boldsymbol{u}_{\text{truth}}$ computed by solving the GLV model introduced in Sec. 1 for the two specific sets of given parameters $\boldsymbol{\theta}$. In the present study, using the reference solution of the GLV model, we generate synthetic data to train the PINN. Still, the impact of different behavior of the dataset on $\hat{\boldsymbol{u}}$ is highlighted in this study in order to test the robustness of the proposed approach to potential issues that may arise when working with real data. To do so, the following situations are considered hereafter:

(1) generating data on a fixed equidistant grid over the time interval $[0, t_{\max}]$ or selecting those times randomly from a uniform distribution;
(2) intentionally reducing the number of available data, to simulate missing or few data in actual experiments;
(3) adding noise to the data, to simulate the noise in actual experiments.

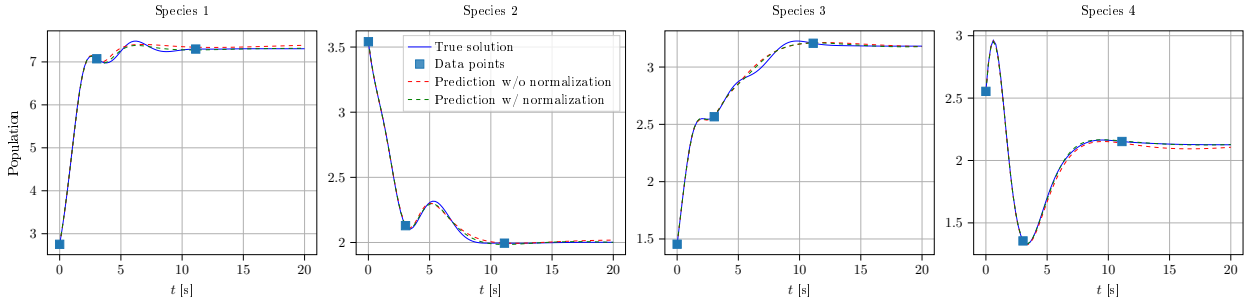
In the following we analyze the accuracy and the efficiency of the PINN with respect to these three situations for the two illustrative examples presented in Sec. 1.2.

3.1. Impact of data sampling strategy

In this section, we focus on the impact of the data sampling on the accuracy of the prediction of the PINN. To begin with, we look at the distribution of the data over the time interval $[0, t_{\max}]$. Two data sets of size $N_{\text{obs}} = 5$ are generated. In the first data set, the data are generated over a uniform grid of the time interval. In the second dataset, the data are generated at times randomly sampled following a uniform distribution in the time interval. The predictions of the PINNs trained with these two datasets and for the two test cases are shown in Figure 7. During the training process, all the collocation points are taken identical and equidistributed over the interval $[0, t_{\max}]$. For the case with 3 species, see Figure 7(a), the errors are $E_{\text{PINN}} = 0.17$ for random



(a) $N_s = 3$.



(b) $N_s = 20$, only 4 species are presented.

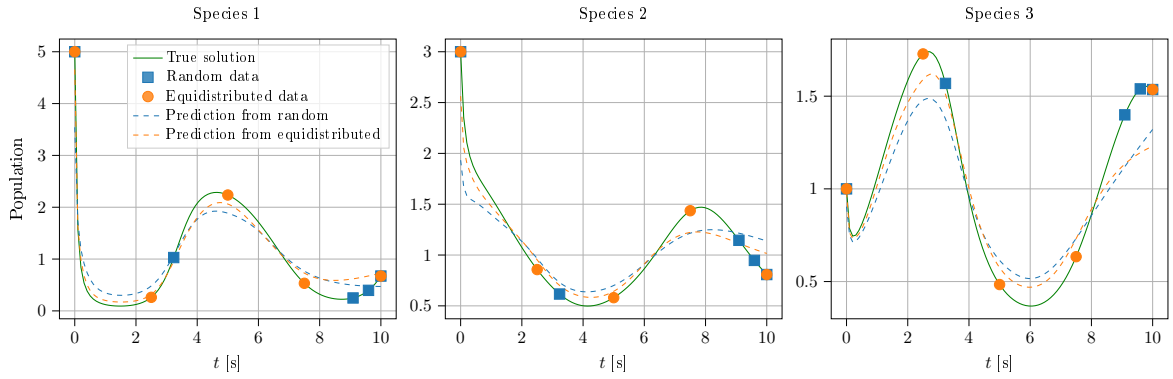
FIGURE 6. Comparison of the PINN predictions with and without time normalization. The full blue line is the true solution of the model, and the dashed lines are the predictions of the two PINNs, with time normalization (green) or without it (red). The appropriate scaling for $\lambda_2^{\mathsf{PINN}}$ is applied.

times and $E_{\text{PINN}} = 0.13$ for equidistant time ones. On the other hand, the case with 20 species results in $E_{\text{PINN}} = 3.56 \cdot 10^{-3}$ for random times and $E_{\text{PINN}} = 3.37 \cdot 10^{-3}$ with equidistant time ones, see Figure 7(b).

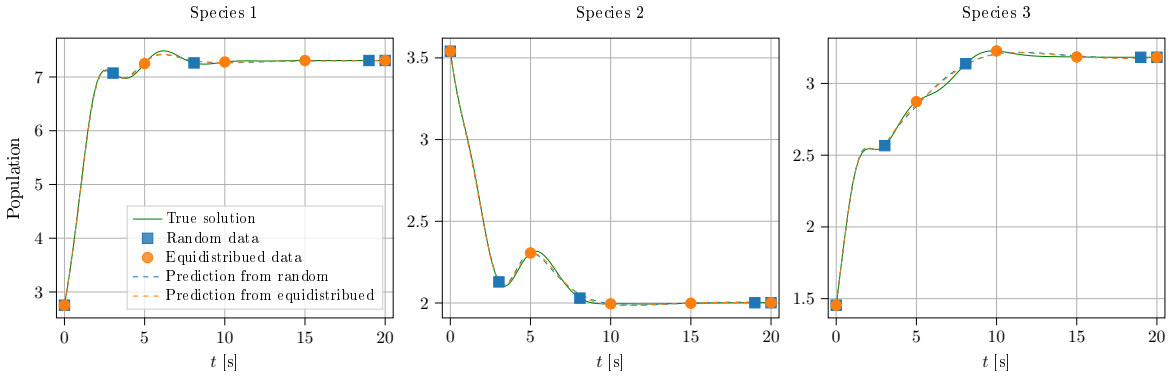
For both cases, the impact of the distribution of the data on the PINN's accuracy is not remarkable: the resulting PINN approximations $\hat{\boldsymbol{u}}$ are similar using different strategies. This outcome is interesting from an experimental point of view, where the sampling strategy can be constrained.

3.2. **Influence of the number of data used in the training on the network's prediction**

First, we focus on the oscillatory example with $N_s = 3$ species, described in Example 1.1. From the model parameters $\boldsymbol{\theta}_3$, two sets of data with various sizes are generated: one with $N_{\text{obs}} = 2$ and $N_{\text{obs}} = 12$. It should be noted that the case $N_{\text{obs}} = 1$ (usually the initial condition) corresponds to the standard framework of PINNs for ODEs, thoroughly investigated in the existing literature [5]. The results of the prediction of the PINN for both sets of data are presented in Figure 8. For a small number of data, the PINN is not able to properly approximate the reference solution and the predicted trajectories are not able to capture the behavior of the true solution. However, as the number of data increases, the prediction becomes more accurate. It is also highlighted by the mean relative errors (9) that is $E_{\text{PINN}}^{(2)} = 0.25$ for the PINN trained with 2 observation data and $E_{\text{PINN}}^{(12)} = 7 \cdot 10^{-2}$ when 12 observations are provided.



(a) Case 1 with $N_s = 3$.

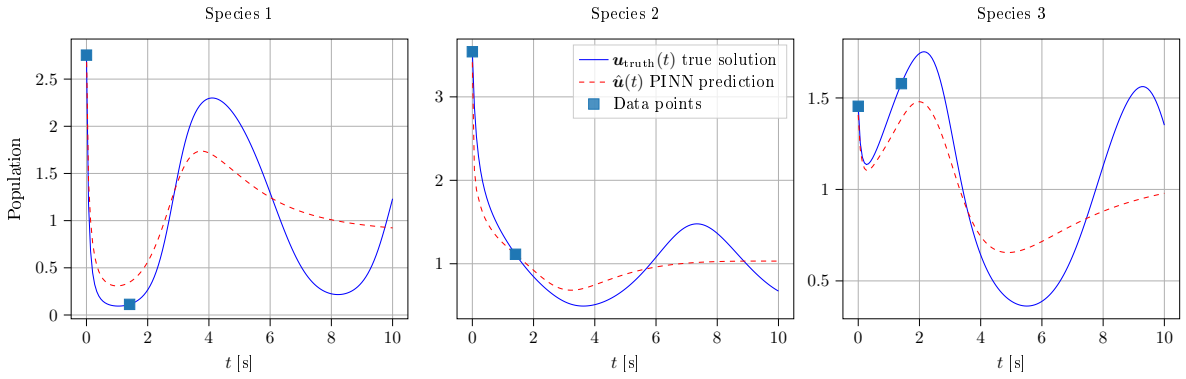


(b) Case 2 with $N_s = 20$, only 3 species are shown. Similar results stand for all other species.

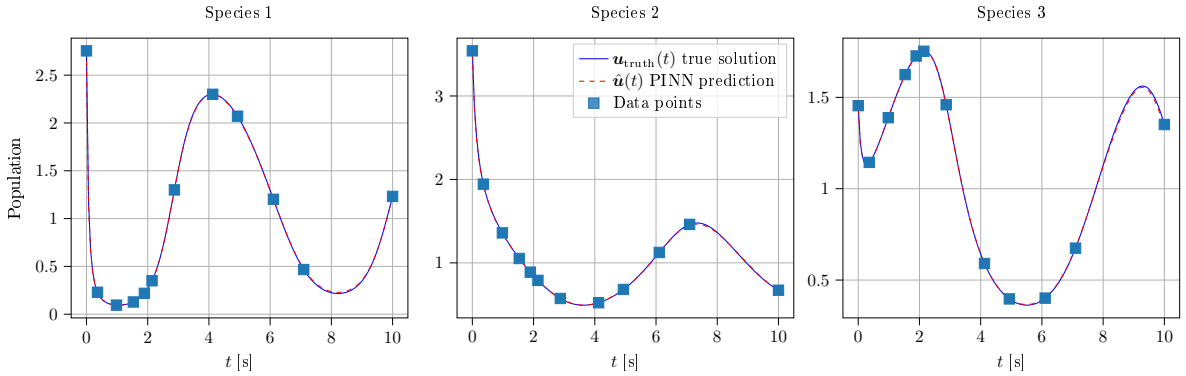
FIGURE 7. PINNs predictions trained with data generated on a fixed equidistant time grid (orange dashed line) and on times randomly selected from a uniform distribution (blue dashed line) over the time interval $[0, t_{\max}]$.

Secondly, we analyze the test case where 20 species are considered, with the convergence toward a stationary state (Example 1.2). The results are presented in Figure 9 for two different training sets: one with $N_{\text{obs}} = 3$ (Figure 9(a)) and one with $N_{\text{obs}} = 11$ (Figure 9(b)). Note that only 4 species are presented. We remark that, with a small number of observations, unlike the oscillatory case, the PINN tends to approximate fairly well the trajectories and fits perfectly the theoretical limit. Precisely, the error is $E^{(3)}_{\text{PINN}} = 5.16 \cdot 10^{-3}$. Doing the same with more data ($N_{\text{obs}} = 11$), see Figure 9(b), we remark that predictions of the PINN are similar to the previous case with $E^{(11)}_{\text{PINN}} = 5.55 \cdot 10^{-3}$. One can conclude that in this case, the number of observations does not influence the PINN approximation and especially the estimation of the stationary state.

In analyzing the results, it is noteworthy that the oscillatory test case exhibits poorer accuracy compared to the stationary counterpart. This disparity can be attributed to the chosen activation function, which inherently struggles to replicate periodic behaviors without additional constraints from more data to guide the predicted trajectories of PINNs. It is worth mentioning that alternative activation functions (*e.g.*, cosine) could successfully capture periodic patterns but might fall short in reproducing stationary behaviors [28]. Given the need to make this activation function choice "offline" before knowing the specific parameters and the system behavior, we have consciously adhered to the hyperbolic tangent choice, prioritizing the reproduction of stationary behavior. This decision aligns with our focus on biological applications that often involve steady-state scenarios.



(a) With 2 data points in the training set.



(b) With 12 data points in the training set.

FIGURE 8. Prediction of the PINN with various numbers of points used for the training set. The population is composed of three species (Ex. 1.1). The full blue line is the true solution of the model, the dashed red line is the model's prediction. The square points are the data used for the training of the PINN.

In specific circumstances, however, alternative activation functions can be employed in the proposed approach to better simulate oscillating behaviors.

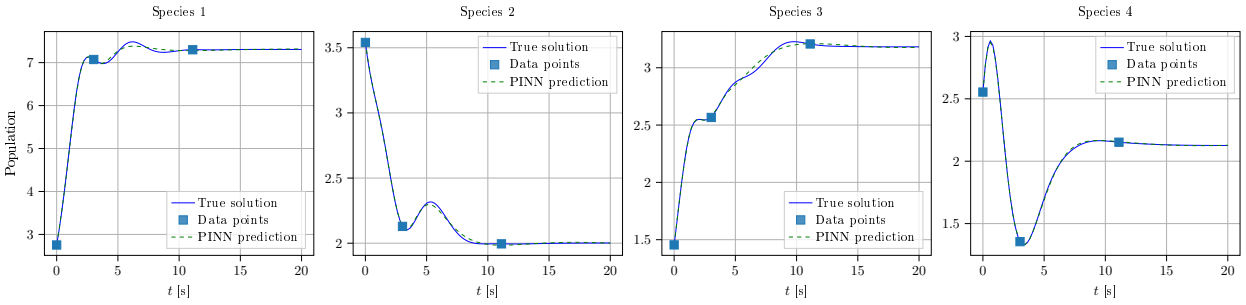
In the oscillatory test case, the inclusion of data points, such as through data loss, is crucial to avoid local minima and prevent constant incorrect predictions, even when no noise is present in the data. Conversely, in the stationary case, while the model can function without such data, adding data points enhances accuracy and reduces computational time, as shown in our preliminary tests. These findings are consistent with previous results reported in [5].

3.3. **Adding noise**

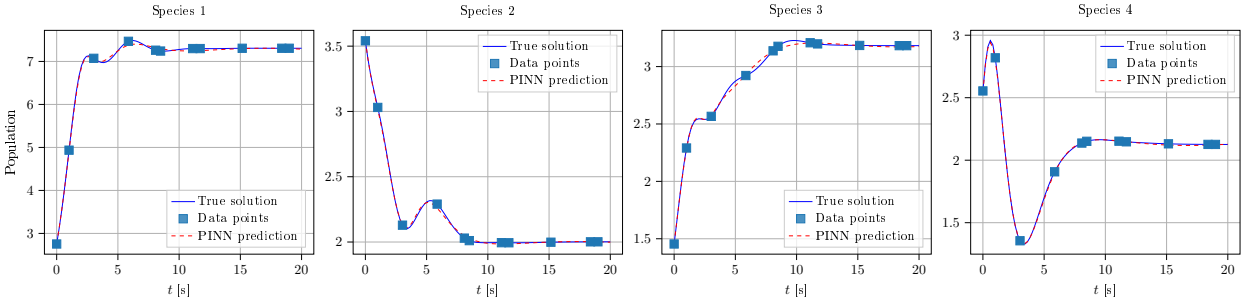
In order to evaluate the proposed methodology on more realistic data, we conducted some tests with noisy synthetic data.

From deterministic generated data y, we define the following multiplicative noise y_{noisy} as:

$$y_{\text{noisy}} = \text{Log-}\mathcal{N}\left(\mu = \ln(y) - \frac{1}{2}\sigma^2, \sigma = \ln(1+\nu^2)\right),$$



(a) $N_{\text{obs}} = 3$.



(b) $N_{\text{obs}} = 11$.

FIGURE 9. Prediction of the PINN with various numbers of points used for the training set N_{obs}, for the test case with $N_s = 20$ (only 4 species are presented). The full blue line is the true solution of the model, predictions of the PINN with and without time-normalization are drawn in dashed lines, green and red respectively.

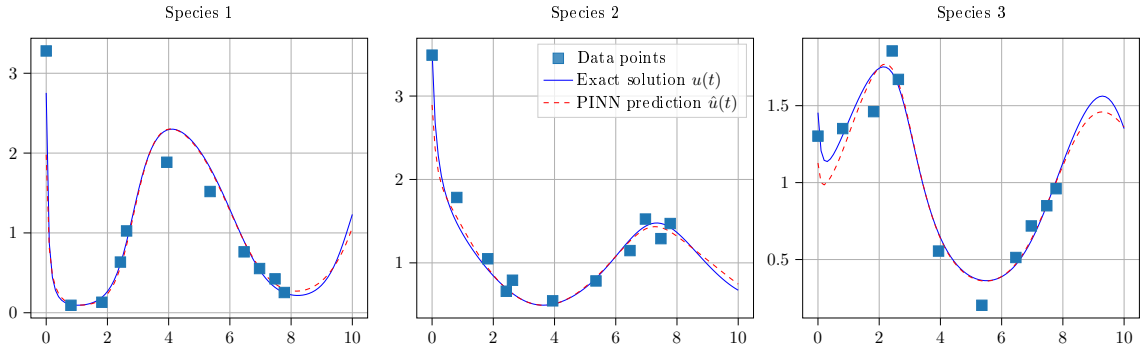


FIGURE 10. Prediction of the PINN with noisy data, for the oscillatory case with $N_s = 3$ species.

where ν is the desired ratio between the standard deviation and the value ($\nu = \frac{\text{std}}{y}$). In the context of noisy data for the GLV model, we select a value for ν between 0.1 and 0.3.

To begin with, we present the results for the oscillatory case with 3 bacterial species, introduced in Example 1.1, see Figure 10. In the figure, the exact solution is still plotted, even if the data are noisy, as we do not expect the predicted trajectory to be equal to the exact solution. The error resulting from the PINN approximation is $E^{(3)}_{\text{PINN}} = 3.18 \cdot 10^{-2}$.

Now we focus on the second example introduced in Example 1.2 with 20 species, see Figure 11. The error resulting from the PINN approximation is $E^{(20)}_{\text{PINN}} = 4.89 \cdot 10^{-2}$.

For both test cases, the predicted trajectories are quite close to the reference solutions and the errors between the PINN approximations and reference solutions are fairly low. This tells that the developed PINN model is robust to noisy data, which is a positive outcome of the method. Note that such results are obtained to a fine-tuning of the hyperparameter λ^{PINN}.

4. Conclusions and perspectives

In conclusion, our utilization of the GLV model within this study has provided valuable insights into the dynamics of microorganism populations. Despite its widespread use, conventional methods may encounter challenges in solving the GLV model due to the intricate behaviors associated with specific parameter sets. By employing PINNs in our research, we successfully simulated the evolution of bacterial species governed by the GLV model. Our proposed approach relies on a loss function that effectively combines the constraints of the physical model with data.

We experimented with various architectures and numerical strategies to enhance the efficiency and accuracy of this methodology. Subsequently, to illustrate the capability of the developed PINN in capturing expected outcomes, we considered two test cases with distinct behaviors. Finally, considering potential experimental applications, we discuss the influence of data, including different sampling, missing data, and noise, on the robustness of this method.

This study on the application of a PINN to the GLV model is a preliminary study to assess the capability of such approach to simulate the evolution of bacterial species. Looking forward, our perspective involves extending this approach into a more complex framework, particularly for the estimation of GLV parameters. Traditionally, algorithms designed for parameter estimation tasks require repeated solving of the model, making computational efficiency a crucial consideration. In this context, the adoption of a fast yet robust method, such as the proposed PINN, holds the potential to be pioneering in the field of parameter estimation for the GLV model. With such methodological developments, we expect to improve the optimization of parameter estimation processes, thus gaining a deeper understanding of microorganism dynamics. However, further investigations, that are out of the scope of this preliminary study, are required to adapt the proposed approach for various parameter values.

Acknowledgements

The authors extend a sincere gratitude to Nicolas Brunel[1] for his mentorship, expertise, and invaluable contributions which greatly enriched this work. The author's contribution was permitted in part by funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (Grant agreement ERC-2017-AdG No. 788191 - Homo.symbiosus). The authors thank the organizers of the CEMRACS 2023 and CIRM for their hospitality.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[2] Eric Aislan Antonelo, Eduardo Camponogara, Laio Oriel Seman, Eduardo Rehbein de Souza, Jean P. Jordanou, and Jomi F. Hubner. Physics-informed neural nets for control of dynamical systems, 2022.

[3] Marie-Claire Arrieta, Leah T Stiemsma, Nelly Amenyogbe, Eric M Brown, and Brett Finlay. The intestinal microbiome in early life: health and disease. *Frontiers in immunology*, 5:427, 2014.

[4] Stephen Baigent. Lotka–volterra dynamical systems. In *Dynamical and Complex Systems*, pages 157–188. World Scientific, 2016.

[1] LaMME, Université d'Evry, Capgemini Invent

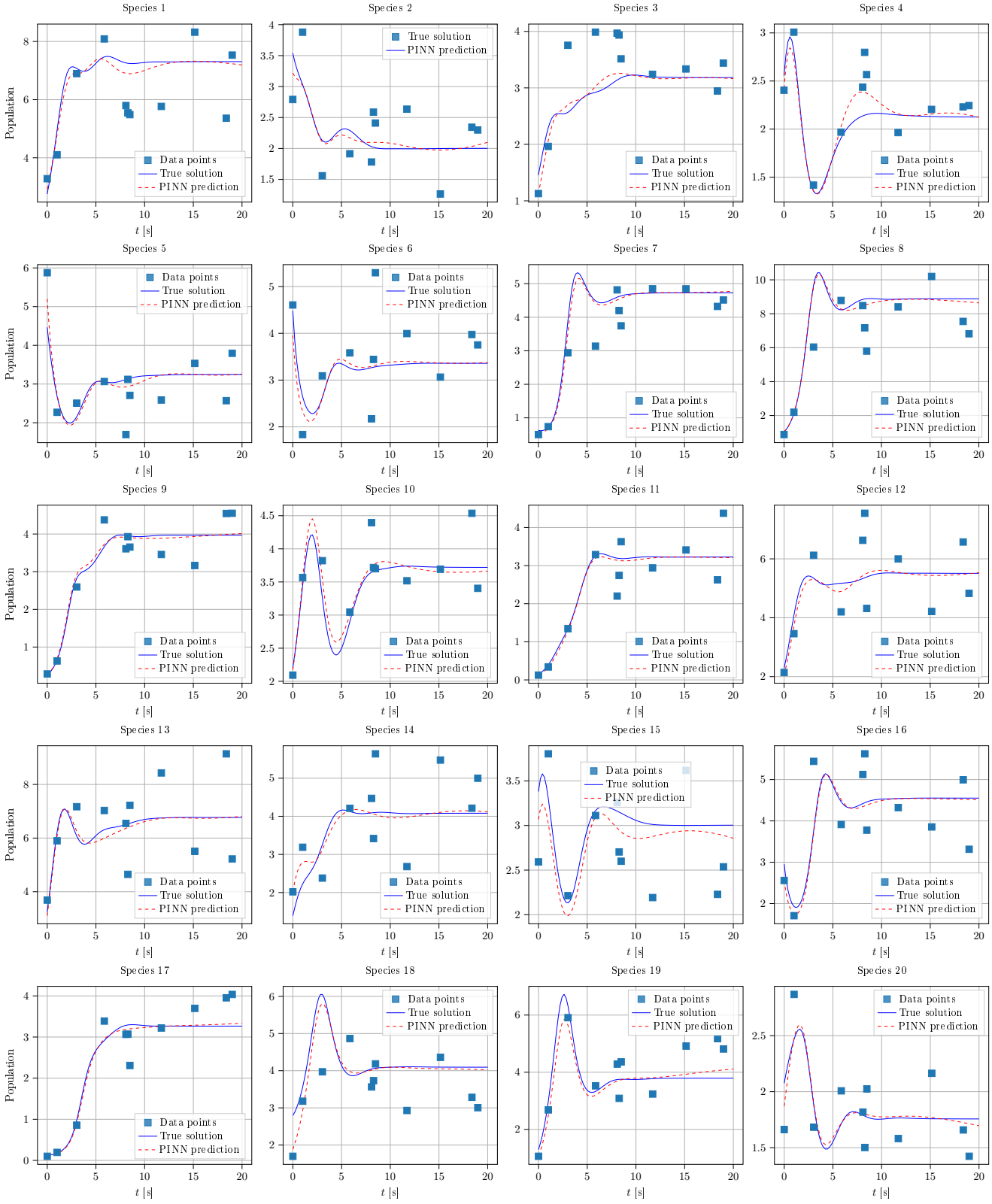


FIGURE 11. Prediction of the PINN with noisy data for the stationary test case with $N_s = 20$ species.

[5] Hubert Baty and Léo Baty. Solving differential equations using physics informed deep learning: a hand-on tutorial with benchmark tests. *arXiv preprint arXiv:2302.12260*, 2023.
[6] Vanni Bucci, Belinda Tzen, Ning Li, Matt Simmons, Takeshi Tanoue, Elijah Bogart, Luxue Deng, Vladimir Yeliseyev, Mary L Delaney, Qing Liu, et al. Mdsine: Microbial dynamical systems inference engine for microbiome time-series analyses. *Genome biology*, 17:1–17, 2016.
[7] Vanni Bucci and Joao B Xavier. Towards predictive models of the human gut microbiome. *Journal of molecular biology*, 426(23):3907–3916, 2014.
[8] Mary I Butler, Sabrina Mörkl, Kiran V Sandhu, John F Cryan, and Timothy G Dinan. The gut microbiome and mental health: what should we tell our patients?: le microbiote intestinal et la santé mentale: que devrions-nous dire à nos patients? *The Canadian Journal of Psychiatry*, 64(11):747–760, 2019.
[9] John F Cryan and Timothy G Dinan. Mind-altering microorganisms: the impact of the gut microbiota on brain and behaviour. *Nature reviews neuroscience*, 13(10):701–712, 2012.
[10] Frank Dondelinger, Dirk Husmeier, Simon Rogers, and Maurizio Filippone. Ode parameter inference using adaptive gradient matching with gaussian processes. In *Artificial intelligence and statistics*, pages 216–228. PMLR, 2013.
[11] Ali Faisal, Frank Dondelinger, Dirk Husmeier, and Colin M Beale. Inferring species interaction networks from species abundance data: A comparative evaluation of various statistical and machine learning methods. *Ecological Informatics*, 5(6):451–464, 2010.
[12] Noah Fierer. Embracing the unknown: disentangling the complexities of the soil microbiome. *Nature Reviews Microbiology*, 15(10):579–590, 2017.
[13] HI Freedman and HL Smith. Tridiagonal competitive-cooperative kolmogorov systems. *Differential Equations and Dynamical Systems*, 3(4):367–382, 1995.
[14] Jonathan Friedman, Logan M Higgins, and Jeff Gore. Community structure follows simple assembly rules in microbial microcosms. *Nature ecology & evolution*, 1(5):0109, 2017.
[15] Dominique Gauguier, Michel Neunlist, Harry Sokol, and Zitvogel Laurence. Microbiote intestinal (flore intestinale). une piste sérieuse pour comprendre l'origine de nombreuses maladies. `https://www.inserm.fr/dossier/microbiote-intestinal-flore-intestinale/`, 2016. Accessed in June 2023.
[16] Georg K Gerber. The dynamic microbiome. *FEBS letters*, 588(22):4131–4139, 2014.
[17] Jérome Harmand. Les modèles de Lotka Volterra Généralisés : intérêts et limites en écologie microbienne. In *Webinaire "Comprendre et valoriser les communautés microbiennes"*, Web, France, September 2020.
[18] Lora V Hooper, Dan R Littman, and Andrew J Macpherson. Interactions between the microbiota and the immune system. *science*, 336(6086):1268–1273, 2012.
[19] JF Jiang. The complete classification of asymptotic behavior for bounded cooperative lotka-volterra systems with the assumption (sm). *Quarterly of Applied Mathematics*, 56(1):37–53, 1998.
[20] Pengzhan Jin, Zhen Zhang, Aiqing Zhu, Yifa Tang, and George Em Karniadakis. Sympnets: Intrinsic structure-preserving symplectic networks for identifying hamiltonian systems. *Neural Networks*, 132:166–179, 2020.
[21] George Karakostas and Istvan Györi. Global stability in job systems. *Journal of mathematical analysis and applications*, 131(1):85–96, 1988.
[22] Pykes Kurtis. The vanishing/exploding gradient problem in deep neural networks understanding the obstacles that faces us when building deep neural networks. `https://towardsdatascience.com/the-vanishing-exploding-gradient-problem-in-deep-neural-networks-191358470c11`, 2020. Accessed in May 17, 2020.
[23] B. Laroche, N. Brunel, D. Goujot, and S. Labarthe. Parameter estimation for dynamical systems using an FDA approach. In *11th International Conference of the ERCIM WG on Computational and Methodological Statistics (CMStatistics 2018)*, Pise, Italy, December 2018.
[24] Alfred J Lotka. *Elements of mathematical biology*. Dover Publications, 1956.
[25] Philippe Marteau and Joël Doré. Le microbiote intestinal. *EMC-Gastro-entérologie*, 12:1–8, 2017.
[26] Jürgen Moser. Periodic orbits near an equilibrium and a theorem by alan weinstein. *Communications in Pure Applied Mathematics*, 29:727–747, 1976.
[27] Ulrich G Mueller and Joel L Sachs. Engineering microbiomes to improve plant and animal health. *Trends in microbiology*, 23(10):606–617, 2015.
[28] Jamshaid Ul Rahman, Faiza Makhdoom, and Dianchen Lu. Amplifying sine unit: An oscillatory activation function for deep neural networks to recover nonlinear oscillations efficiently. *arXiv preprint arXiv:2304.09759*, 2023.
[29] M. Raissi, P. Perdikaris, and G.E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
[30] J. O. Ramsay, G. Hooker, D. Campbell, and J. Cao. Parameter estimation for differential equations: a generalized smoothing approach. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(5):741–796, 2007.
[31] Philippe Sansonetti. La relation hôte-microbiote : une insondable symbiose ? *Lett. Du Coll. Fr.*, 32:14–15, October 2011.
[32] Nicola Segata, Jacques Izard, Levi Waldron, Dirk Gevers, Larisa Miropolsky, Wendy S Garrett, and Curtis Huttenhower. Metagenomic biomarker discovery and explanation. *Genome biology*, 12:1–18, 2011.

[33] Richard R Stein, Vanni Bucci, Nora C Toussaint, Charlie G Buffie, Gunnar Rätsch, Eric G Pamer, Chris Sander, and Joao B Xavier. Ecological modeling from time-series inference: insight into dynamics and stability of intestinal microbiota. *PLoS computational biology*, 9(12):e1003388, 2013.

[34] Matthew Tancik, Pratul Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in neural information processing systems*, 33:7537–7547, 2020.

[35] Elizabeth Thursby and Nathalie Juge. Introduction to the human gut microbiota. *Biochemical journal*, 474(11):1823–1836, 2017.

[36] V. Volterra. *Variazioni e fluttuazioni del numero d'individui in specie animali conviventi*, volume 2. Societá anonima tipografica" Leonardo da Vinci", 1927.

[37] V. Volterra and M. Brelot. *Leçons sur la théorie mathématique de la lutte pour la vie*. Paris : Gauthier-Villars, 1931.

[38] Philippe Wenk, Alkis Gotovos, Stefan Bauer, Nico S Gorbach, Andreas Krause, and Joachim M Buhmann. Fast gaussian process based gradient matching for parameter identification in systems of nonlinear odes. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pages 1351–1360. PMLR, 2019.

[39] Bohra Yash. The challenge of vanishing/exploding gradients in deep neural networks. `https://www.analyticsvidhya.com/blog/2021/06/the-challenge-of-vanishing-exploding-gradients-in-deep-neural-networks/`, 2021. Accessed in June 2021.